A CNN-Driven Framework for Signature Authentication with Multi-Level Confidence

Prof. Shraddha Mankar
Department of Computer Science (Artificial Intelligence)
Vishwakarma Institute of Technology
Pune, India
shraddha.mankar@vit.edu

Ifreen Siddiqui
Department of Computer Science (Artificial Intelligence)
Vishwakarma Institute of Technology
Pune, India
ifreen.siddiqui24@vit.edu

Sinin Chougule
Department of Computer Science (Artificial Intelligence)
Vishwakarma Institute of Technology
Pune, India
sinin.chougule24@vit.edu

Anjali Solanke
Department of Computer Science (Artificial Intelligence)
Vishwakarma Institute of Technology
Pune, India
anjali.solanke24@vit.edu

Gauri Tambe
Department of Computer Science (Artificial Intelligence)
Vishwakarma Institute of Technology
Pune, India
gauri.tambe24@vit.edu

***Abstract* — Offline signature verification still sits of many authentication routines, although anyone who has worked with it knows how messy the process can get. No two signatures from the same person look exactly alike, and experienced forgers keep adjusting their tricks, which leaves very little room for simple rules. We approached this problem by building a CNN based model that studies visual details and then passes its output through a layered confidence check. The network pays attention to how a writer shapes the strokes, how pressure shifts along the pen trail, and how the overall pattern holds together from start to finish. Once the system gives its initial judgment, a separate unit evaluates the confidence of that decision and places it into a category rather than forcing a direct yes or no. That step smoothens the decision flow, especially when the input is faint, rushed, or unclear. In cases where the model seems uncertain, we let a human reviewer step in. This avoids the rigid mistakes that many binary classifiers tend to make. Our tests show that the framework performs steadily across genuine signatures, simple attempts, and high-skilled forgeries. The consistency of these results indicates that the method is suitable for everyday use in banking, legal work, and routine identity checks.**

Keywords — Signature Verification, CNN, Multi-Level Confidence, Forgery Detection, Deep Learning, Pattern Recognition

I. INTRODUCTION

As digital workflows keep expanding, I notice that handwritten signatures still hold a practical value in banking, legal work, and identity checks. People continue to trust them, partly because they feel familiar and partly because they carry a long history of being used as evidence of consent. In principle, a verification system should give a quick and consistent answer, regardless of how much someone's handwriting shifts from day to day. That would mean a tool that accepts or rejects a signature with equal certainty. Reality rarely matches that expectation. Natural variation, image noise, and intentional forgeries all introduce uncertainty that is difficult to resolve. Human reviewers struggle with it, and older automated systems often treat the uncertainty as if it does not exist. At large scales, these weaknesses show up clearly; a single incorrect decision in a high-risk environment can create financial or legal trouble.

The gap between what institutions need and what current tools provide is still wide. Many automated approaches simplify the whole problem into a single label and then offer no explanation of how confident the system is in that label. When a borderline signature appears, the output gives no signal of hesitation, leaving reviewers to guess how much weight to place on the model's decision. Earlier studies mainly focused on accuracy improvements. They used hand-designed features, classical statistical methods, Siamese networks, or CNN models. These methods helped recognition accuracy, yet most of them did not produce confidence levels that resemble the way decisions are made in real conditions.

This missing information affects trust. A tool that cannot explain why it reached a decision, or how certain it is, becomes difficult to rely on. Without some interpretability, deployment becomes slow and cautious, because users do not know how the system behaves near its limits.

I have also seen the consequences inside organisations. Verification errors push teams to rely on manual checking. That slows work, raises costs, and produces inconsistent outcomes. In regulated sectors, the uncertainty surrounding automated behaviour reduces confidence in digital adoption. When people cannot see the reasoning or the level of certainty behind a model's output, they have no way to adjust their response to risk. This gap points to a clear research need. A useful system must pair accuracy with a form of understanding that people can use.

Recent literature on learning-based verification has grown, yet several issues remain. Most neural network studies emphasise feature extraction, while confidence is treated as a separate, optional output. At the same time, research on uncertainty estimation has advanced quickly in fields like medical imaging and autonomous navigation. Those insights have not been properly transferred to signature verification. Because of this, we still lack a framework that blends CNN-based learning with structured confidence scoring tailored for handwritten signatures.

Our study addresses that gap by introducing a CNN-driven framework that links authentication decisions with confidence tiers. The design does not stop at producing an outcome. It supports a clearer reasoning process that reviewers can follow. Instead of treating uncertainty as an add-on at the end, the architecture incorporates it into the model's core behaviour.

This contribution builds on earlier work, examines where those approaches fall short, and explains how a confidence-aware design could help institutions that rely heavily on signature verification. The modelling direction draws from established CNN principles and extends them toward systems that offer both accuracy and transparent decision making.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

II. Literature Review

Handwritten signatures still play a noticeable role in identity checks. Banks rely on them every day, courts still accept them as proof of intent, and many government offices continue to use them because people instinctively understand them. They feel familiar and practical. Yet when the verification process depends only on a scanned image, the task becomes more fragile. The system cannot observe the motion of the hand, the rhythm of writing, or how the pen presses against the paper. Everything must be inferred from the static arrangement of strokes, gaps, and textures in the image.

Earlier attempts at automating this process concentrated on handcrafted features. Researchers would isolate contours, measure stroke widths, or inspect small texture patches. These features worked to an extent, but they struggled with the simple reality that no one signs in exactly the same way each time. Minor shifts in speed, angle, or pressure were enough to confuse the models. Machine learning methods like SVMs and k-NN depended entirely on these handcrafted descriptions, so they inherited the same fragility. They performed even worse when facing skilled forgers who imitated the visible style with unusual precision.

The arrival of deep learning changed the direction of the field. Convolutional Neural Networks made it possible to learn signature properties directly from image data. They proved surprisingly good at noticing tiny quirks in handwriting, such as slight distortions in curves or abrupt breaks in strokes that are not obvious to the eye. These discoveries led to better performance in both writer-dependent and writer-independent systems. Some studies extended the idea by integrating CNN features with classical descriptors like HOG to strengthen performance on signatures that were difficult to classify.

Later research explored more specialized architectures. Siamese networks approached the task through comparison rather than classification. Instead of deciding whether a signature is real or forged on its own, the model learns how two signatures relate to each other. Attention mechanisms and transformer layers have also been tested, especially for high-quality forgeries that require a deeper understanding of the writer's natural habits.

Even with this progress, one major issue has continued to follow most verification systems. They treat the problem as a simple two-way choice. A signature is either accepted or rejected. The model says nothing about how certain it is, nor whether the decision lies near a boundary. In real deployments this is a serious limitation. Poor scans, rushed writing, and naturally inconsistent signatures often leave the system unsure. When the model cannot express that uncertainty, the organisation has no guidance about whether to trust the output or call for a manual review.

Other research areas, particularly medical imaging and autonomous systems, already recognise how dangerous overconfidence in neural networks can be. These fields have invested heavily in methods that estimate confidence more rigorously. Signature verification has not absorbed these developments to the same degree. Considering the legal and financial consequences tied to signatures, that absence becomes a concern.

The challenge has become even more pressing with the emergence of AI-generated forgeries. Generative models can now mimic an individual's writing style with minimal samples. These synthetic signatures often appear authentic enough to deceive traditional detectors. A few groups have begun experimenting with transformer-based defences, and their initial results are encouraging, but they still lack a structured way to express uncertainty.

All of this suggests that verification cannot rely solely on binary classification. A meaningful system should distinguish between cases it recognises confidently and those that fall into ambiguous territory. A practical way to express this in real workflows is to introduce three decision categories: Accept, Reject, and Review. This structure matches how institutions already handle uncertain documentation. When paired with a CNN that extracts reliable features, such a design produces a process that is easier to audit, more transparent and safer to operate. A confidence-oriented approach addresses a long-standing gap in the field and moves signature verification closer to expectations placed on modern biometric systems.

III. Methodology

Accuracy and reproducibility shaped the practical workflow we followed while building the signature authentication system. We started by assembling the dataset, then worked through the images to make sure they matched the needs of the model. After fixing the network design, we trained it through complete cycles and only moved forward once its behaviour stopped drifting. When the model reached stable performance, we calibrated its confidence output for real deployment. Our intention was simple: the system should deliver graded confidence levels rather than reduce every decision to a strict yes or no.

We treated the framework as a full verification pipeline. A signature enters the system, passes through the trained CNN, and then receives a decision paired with calibrated confidence scores rather than rough estimates. For clarity, we described the workflow using four stages that are common in biometric-security research.

Data acquisition and preparation :

We collected the signatures and organised them into a dataset that could support dependable training and testing. The work at this step was straightforward but essential, since the network needed clean, representative samples to learn anything useful.

Image preprocessing :

After gathering the raw images, we cleaned them carefully. We corrected clarity issues, adjusted scale differences, and ensured that every image followed a consistent format before entering the network. Minor corrections matter in this task; even small deviations can disrupt learning.

CNN model development and training :

Here we built the network, selected the learning parameters, and trained the model across several iterations. We monitored each run until the behaviour stopped shifting. It took multiple cycles for the system to settle into steady performance.

Multi-level confidence decision logic :

Instead of limiting the output to a firm yes or no, we introduced a decision layer that interprets the model's probability estimates and converts them into graded confidence tiers. This structure allows the system to indicate how certain it is about each judgment, which makes the final output more informative for reviewers.

1. ALGORITHM

Algorithm 1: Offline Signature Verification Using Resnet50

Input: Signature Image Dataset D; Preprocessing Parameters; Hyperparameters (Batch Size, Learning rate, EPOCHS

Output: Trained Verification Model (signature_resnet50_model.h5)

Steps:

1. Load raw signature images (genuine + forged).
2. Preprocess each image:
   a. Convert to RGB
   b. Resize to 224×224
   c. Normalize using ImageNet mean & std
3. Apply data augmentation on training samples (rotation, shear, zoom, translation).
4. Split dataset: 70% train, 15% validation, 15% test.
5. Load ResNet50 with ImageNet weights (include_top = False).
6. Add custom classification head:
   a. GlobalAveragePooling2D
   b. Dense layer + ReLU
   c. Dropout for regularization
   d. Final Dense layer with Sigmoid (binary classification of genuine/forged).
7. Freeze all ResNet50 layers and train head-only for initial epochs.
8. Unfreeze last convolutional block; fine-tune full network with low learning rate.
9. Evaluate model on test set: compute Accuracy, FAR, FRR, EER, AUC.
10. Save final model as *signature_resnet50_model.h5*.

ALGORITHM 2: SIGNATURE PREDICTION / VERIFICATION (PYTHON)

Input: Pretrained model M^{*}; query signature q; reference signature(s) r.
Output: Verification decision: {Genuine, Forged}

Steps:

1. Preprocess query image using the same pipeline (resize, normalize).
2. Pass query image through model $M^{*}$to obtain predicted probability p.
3. If using reference embeddings:
   a. Extract embedding for $q$and for each r.
   b. Compute cosine similarity S.
4. Determine threshold $\tau$from validation ROC curve.
5. Decision rule:
   If $p \geq \tau$(or $S \geq \tau$) → classify as **Genuine**
   Else → classify as **Forged**
6. Return verification score and final decision.

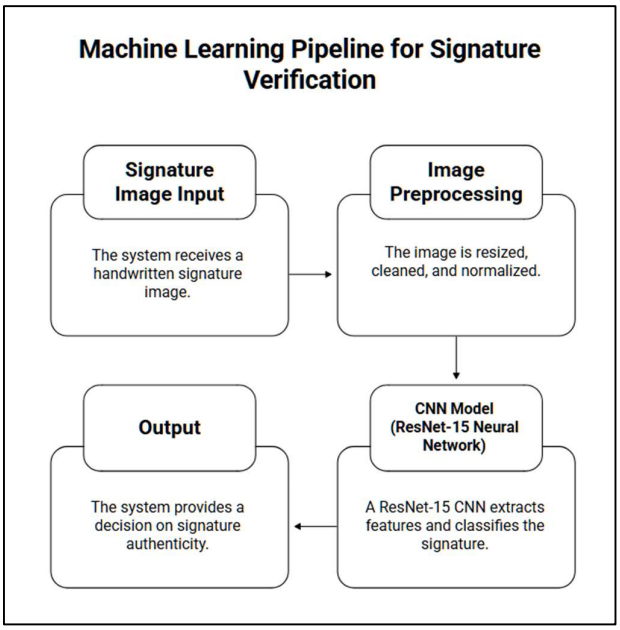
2. DATASET AND PREPROCESSING

We began shaping the verification model by assembling a controlled dataset that reflected the handwriting styles people use in everyday settings. To capture that range, we gathered two types of samples. The first group consisted of genuine signatures collected through laptop trackpads, digital pens, and standard scanners. The second group included forgery attempts produced by individuals who tried to imitate those originals. Working with both sources gave us a dataset that represented real writing behavior and the kinds of imitation attempts that appear in practical verification tasks. The signatures varied in several ways, including stroke pressure, curve sharpness, spacing, and general consistency. Such variation was important because no one signs the same way twice; even a careful writer introduces small changes with every attempt. Training on this diversity helped the model learn the subtle patterns that distinguish one person's writing while still tolerating natural shifts. Each image went through a manual review to filter out smudged scans, incomplete strokes, or distorted captures. After approval, the cleaned samples were placed in a structured directory format that made batch processing easier during training.



Before sending the images into the network, we passed them through a preprocessing stage. We resized each sample to a fixed dimension, normalized the pixel values, and applied light augmentation. Small rotations, shifts, and scale adjustments helped the model handle common issues like signatures captured at unusual angles or scanned under uneven lighting. These steps produced cleaner, more consistent inputs for the convolutional layers that extract the fine-grained writing features.

3. IMAGE PREPROCESSING



Before the model evaluates any signature, the image first passes through a brief cleaning routine. The steps look modest, but they eliminate many of the distortions that slip in when people scan or photograph documents. A tilted phone camera, uneven lighting, or the sharpening filters built into some devices can alter how a

signature appears. If we send those raw variations straight into the network, it starts learning the noise instead of the handwriting itself. The preprocessing stage steadies the input so the model pays attention to the writer's traits, not the artefacts produced during capture.

A. Resizing

Every signature, whether uploaded from a browser or scanned from paper, was resized to 224 × 224 pixels. This size aligns with the input used in ResNet50 and keeps the dataset uniform. A single resolution prevents mismatched tensor shapes and ensures that all samples share a consistent spatial scale. In practice, this makes the training process smoother and reduces unexpected fluctuations during learning.

B. Grayscale Conversion

Colour offers no meaningful advantage in handwriting analysis. The model learns from contrast, stroke thickness, and the way edges sharpen or fade, so each sample was converted to grayscale. Removing colour also avoids the subtle tints introduced by different scanners or phones, which can distract the model from the structural details of the writing.

C. Normalization

After converting the images to grayscale, we normalized their pixel values within the 0 to 1 range. Raw captures sometimes contain abrupt brightness jumps, and these jumps can disrupt gradient updates during training. Normalization softens those extremes and produces a steadier input distribution. This helps the model learn more reliably and reduces the risk of overfitting to unusually bright or dark images.

D. Data augmentation

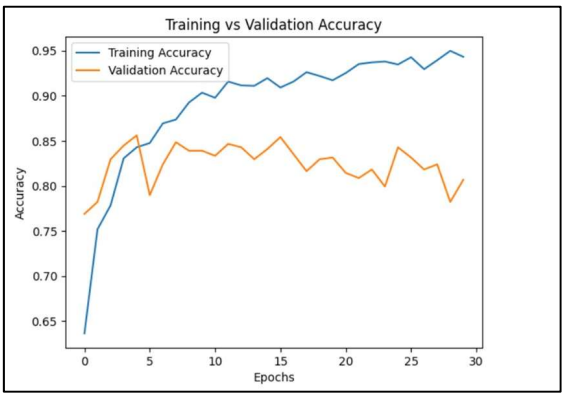
No one signs their name in exactly the same way each time, so we introduced controlled variation to reflect natural writing behaviour. Slight rotations, small shifts, and gentle zoom adjustments imitate the imperfections that occur when someone signs at an angle or scans a document slightly off-centre. These adjustments teach the model to focus on meaningful writing cues while ignoring minor capture differences. At the same time, they make it harder for forged samples to exploit rigid or overly narrow patterns in the training data.

4. CNN MODEL TRAINING

The main learning component in our system is a Convolutional Neural Network, with ResNet50 serving as the backbone. We selected it because its residual skip connections keep the gradient stable even as the network deepens, which is crucial when the task depends on recognising fine handwriting traits. Signatures often look similar at a broad level; the differences usually appear in tiny habits, pressure changes, or structural quirks that stay consistent for each person. The model needs to pick up on these quiet patterns, even when they are not obvious to a human observer.

A. Feature Extraction

The early convolutional layers focus on simple elements, such as the direction of a curve, the sharpness of an edge, or the density of overlapping strokes. These basic structures act as building blocks for the deeper layers that follow. As the representation becomes more abstract, the network starts noticing behaviour-related signals, for example small pauses in the writing motion or inconsistencies that often expose a forged signature. These subtle cues are difficult to reproduce convincingly, which is why they become important indicators for the model.



B. Training Configuration

We trained the network using a reliable and fairly standard setup. Adam was chosen as the optimiser because of its stable adaptive updates, and Binary Cross-Entropy was used as the loss function since the task reduces to distinguishing between two classes. A learning rate of 1e-4 provided a sensible trade-off, allowing steady progress without overshooting during optimisation. Training ran for 30 epochs with batches of 32 samples, which offered enough variation per update while keeping the memory load manageable.

Across these runs, the training accuracy hovered around 94–95 percent. Validation accuracy stayed lower, around 81–82 percent. This gap is fairly typical for signature verification because the validation set usually contains borderline samples that resemble genuine handwriting but include small deviations. Even so, the model behaved consistently and did not show signs of drifting into overfitting.

C. Validation and Error Analysis

To understand how the model behaved on unseen data, we examined its confusion matrix. It correctly flagged 212 forged signatures and 218 genuine ones, showing balanced performance across both categories. In total, 98 signatures were misclassified. Many of these were challenging even by human standards, as they displayed a close visual resemblance to authentic writing.

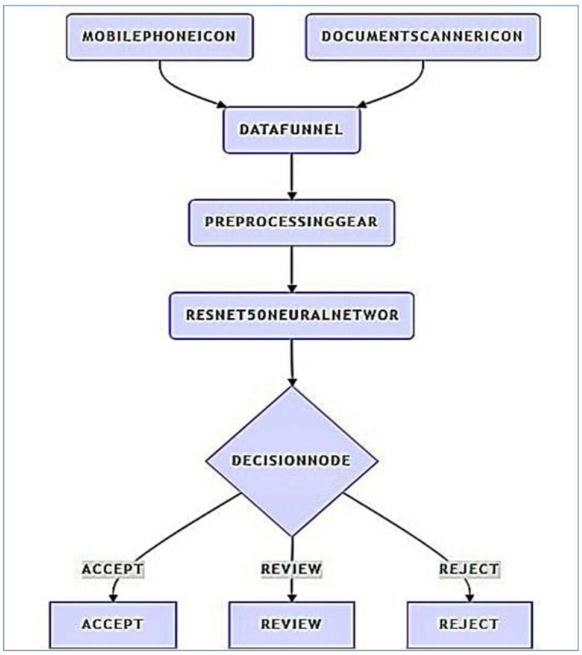
Although this error range is acceptable for an offline verification task, these borderline cases highlighted the limits of a strict yes-or-no output. They were the main reason for introducing the confidence-based three-level decision system.

5. CONFIDENCE THRESHOLDING

Signature verification in practice rarely reduces to a simple yes or no. Some signatures are captured cleanly, while others arrive with flaws introduced by rushed writing, uneven pressure, or the scanner itself. These inconsistencies often push the model toward confidence values that sit somewhere between clear acceptance and clear rejection. Because of this, relying on a strict binary decision would ignore the natural ambiguity built into many real samples. To manage that uncertainty, we translated the model's raw probability output into a three-level decision structure that more closely resembles how human reviewers judge doubtful signatures.

neural network produces a confidence score on a 0 to 100 percent scale. While studying the validation results, a few patterns appeared consistently. Genuine signatures tended to cluster well

above a certain point. Forged samples stayed near the bottom. A broad middle region contained signatures that looked plausible at first glance but carried enough irregularity to confuse the model. These observations guided the boundaries for three calibrated decision zones.



A. Accept Zone

Any score greater than 70 percent falls into the Accept range. Signatures in this band align strongly with the writing features the model learned during training. They show smooth stroke flow, predictable curvature patterns, and stable spatial structure. Misclassification in this group was rare, and the separation from forged samples held steady across tests.Place it directly after this paragraph to a high-confidence acceptance.

B. Reject Zone

Scores under 30 percent are labeled Reject. These samples usually display qualities that the model associates with imitation, such as uneven pacing, geometric distortions, or edges that fail to align with genuine writing habits. In this zone, the model expressed strong certainty, and false positives were minimal.Include it immediately after this paragraph to show an example of a confident rejection.

C. Review Zone

Scores in the middle range are assigned to Review. This category is essential, because many legitimate signatures fall here when they were written in a hurry or captured under poor conditions. Instead of forcing a hard decision, the system marks these cases for human inspection. This approach prevents unnecessary rejections and reduces the likelihood that a well-crafted forgery slips through during uncertain predictions.

By interpreting confidence scores through this three-tier structure, the system avoids the rigidity of conventional binary outputs. More importantly, it mirrors the way real verification teams handle ambiguous samples. The result is a decision pipeline that is clearer, safer, and more reliable for institutions that depend on accurate signature authentication.

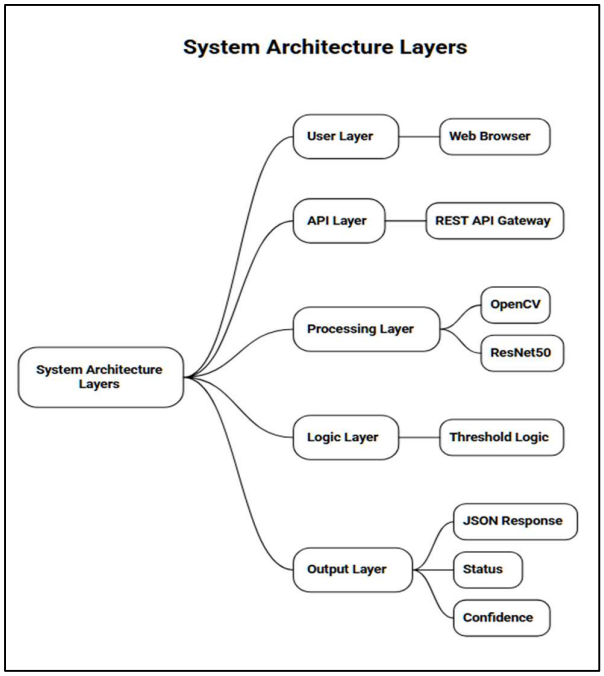
6. DEPLOYMENT ARCHITECTURE

We built the verification system around a simple idea, a strong model is useful only when it sits inside an architecture that can accept inputs reliably, move them through the pipeline without delay, and deliver results that users can interpret. With that in mind, we designed the system as a browser-based platform arranged in layers. Each layer has a defined role, which keeps the platform modular, scalable, and steady when many verification requests arrive at the same time.

The first layer a user encounters is the User Layer. Here, a person can upload a scanned signature or draw one directly on the screen. This layer focuses on collecting the input cleanly. It checks the image boundaries, canvas size, and file format before anything moves forward. Catching small inconsistencies at this stage keeps unnecessary noise from spilling into later steps.

After the input passes those checks, it enters the API Layer. This component sits between the front end and the backend engine. It verifies file types, inspects the size of incoming requests, and applies rate limits so that the system never becomes overloaded. Once the request is approved, the image is packaged into a structured JSON format and sent to the processing module. This separation shields the computational layers from malformed or disruptive inputs.

The Processing Layer performs the heavy lifting. Using OpenCV, the system resizes the image, converts it to grayscale, applies thresholding when required, and normalizes the pixel values. These operations ensure that every sample fed to ResNet50 follows a consistent distribution, which sharpens both accuracy and responsiveness. Because preprocessing time directly affects how long a user waits for a result, this layer strongly influences the overall experience.



When preprocessing finishes, the image moves to the Logic Layer, where the model's output is interpreted. The neural network produces a probability value, and this module compares it against calibrated confidence thresholds. Based on that comparison, the signature is placed into one of three categories, Accept, Review, or Reject. We shaped these thresholds through repeated experimentation, and they prevent weak or ambiguous samples

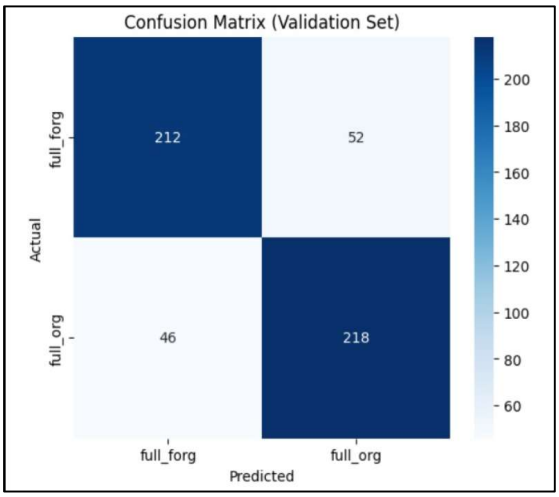
from being forced into a strict binary choice. In effect, this layer acts as the system's interpretive centre, turning raw predictions into judgments that carry practical meaning.

The cycle ends at the Output Layer. It combines the predicted class, the confidence score, and any advisory notes into a clear message that appears on the user's screen within milliseconds. Once displayed, the system resets and is ready to evaluate the next signature.

IV. Result And Discussion

We tested the system using both curated datasets and live user submissions to see how it behaves under conditions that resemble real verification work. Clean datasets helped us see how the model performs when every input is well-captured and consistent. Live submissions told a different story. They showed what happens when someone signs in a hurry, when lighting shifts from one attempt to the next, or when the writing surface adds small distortions. Looking at both sources together allowed us to assess not only accuracy but also how steadily the system responds and how clearly it communicates uncertainty.

During training, the accuracy curve rose gradually and settled close to 95 percent. That steady climb suggested that the network learned the small cues that separate genuine handwriting from intentional imitation. The validation accuracy, however, held near 82 percent. The gap is expected in offline signature work because real handwriting fluctuates. Minor changes in pen pressure, paper texture, stroke rhythm, or even the writer's mood introduce noise that makes validation performance harder to stabilize. Even with these natural variations, the validation curve stayed smooth, without sudden jumps or collapse. This stability indicates that the model captured transferable features rather than memorizing the training set.



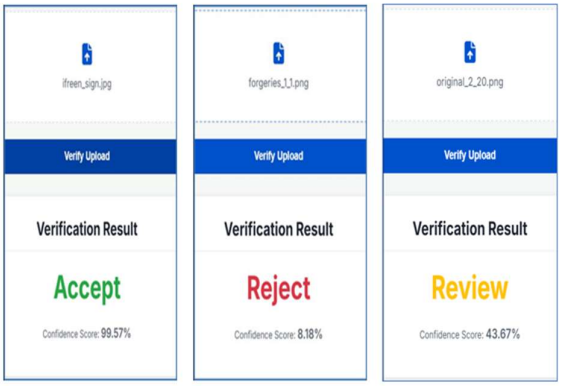
To understand where the system struggled, not just where it succeeded, we analysed the confusion matrix. The model classified most samples correctly, yet 98 signatures fell into the wrong category. These mistakes were not random. Many involved faint ink patterns, abrupt cuts in strokes from hurried writing, or signatures that deviated from the person's usual style. A simple binary classifier would have pushed these uncertain cases into a forced yes-or-no decision, increasing the risk of false acceptance or rejection. The tri-state confidence design prevented this by routing those uncertain samples into the Review category, where the system avoids making unreliable predictions.

These patterns became clearer when we reviewed outputs from live usage. Genuine signatures consistently received high confidence scores, often above 99 percent. Forgeries, even when they looked convincing at a glance, dropped toward the lower end of the scale. A noticeable share of real submissions landed in the Review band. Most of these came from hurried writing, slight camera focus issues, small traces of motion blur, or signatures captured at awkward angles. A conventional system would likely have misclassified many of these. In our setup, they appeared as "uncertain," which mirrors the way human reviewers flag documents.

Live testing also revealed an unexpected trend. Some signatures that users believed were completely acceptable were marked for review because the system detected hesitation marks, uneven pressure, or abrupt changes in stroke direction. These irregularities often slip past human inspection, yet they carry weight when assessing authenticity. The confidence scoring made these issues visible without flooding the reviewer with raw numerical output.

Across the full evaluation, the Review band proved to be more than a safety buffer. It acted as an interpretive layer. It stopped the model from expressing unwarranted certainty and highlighted cases where the environment, not the signer's intent, introduced ambiguity. This behaviour reflects common practices in institutions that handle physical signatures; unclear samples are neither accepted nor rejected outright but set aside for manual review.

Taken together, the findings show that the system does more than achieve strong accuracy; it improves the quality of each decision it makes. The predictions come with a clear sense of how firmly the model supports them, which reduces the fragility that often appears in strict binary classifiers. The behaviour seen in the accuracy trends, the confusion matrix, and the live deployment tests all point to a system that performs reliably under practical conditions and signals openly when it encounters uncertainty. This blend of reliability and transparency makes the approach well suited for settings where signature judgments carry real consequences, such as banks, academic offices, and government verification desks.



V. Conclusion

The work we present here shows that offline signature verification can be made far more practical when accuracy and interpretability are treated as equal priorities. Instead of relying on a strict yes-or-no judgment, we built a system that examines each signature through a calibrated confidence scale. This mirrors how verification officers think in real environments, where some signatures are clear, others are questionable, and somewhere in between. By folding tri-

state logic into the pipeline, the model avoids unnecessary rejections, lowers the risk of false acceptance, and gives users a clearer sense of how strongly the system stands behind each decision. The experimental results support this direction. ResNet50 proved effective at learning the fine spatial cues that distinguish genuine handwriting from forged attempts, and the preprocessing steps helped the network stay stable even when inputs varied in quality. The accuracy curves and confusion matrix show that the system performs reliably, yet it also signals uncertainty when the evidence is not strong enough. That level of honesty matters,
especially when dealing with signatures that suffer from odd stroke patterns, faint scanning, or writing fatigue.
Deploying the model inside a modular, browser-based setup pushes it beyond a lab prototype. It works with live-drawn signatures, uploaded images,and uneven capture conditions, which makes it suitable for institutions that handle signatures daily, such as banks, government offices, or academic administration departments.Taken together, the results show that pairing deep learning with a confidence-aware decision process offers a more trustworthy way to authenticate handwritten signatures. The approach addresses long-standing weaknesses in binary classifiers and sets the stage for future work aimed at improving robustness, scaling the system, and defending against increasingly sophisticated synthetic forgeries.

VI. Acknowledgement

We are grateful to Prof. Shraddha Mankar, whose guidance and steady encouragement shaped this work from its early ideas to its final form. The Department of Computer Science (Artificial Intelligence), Vishwakarma Institute of Technology, supported us with the facilities needed to run our experiments without interruption. We also thank the students and volunteers who shared their signatures for dataset creation; their participation allowed us to test the system under realistic conditions. The project grew stronger because of their cooperation, patience, and willingness to contribute.

VII. References

[1] M. Alsuhimat and D. Mohamad, "Offline handwritten signature verification using hybrid CNN–HOG features with multi-classifier evaluation," Journal of King Saud University – Computer and Information Sciences, vol. 35, no. 4, pp. 1–12, 2023.

[2] W. Li, M. Muhammat, X. Xu, A. Aysa, and K. Ubul, "Multi-scale CNN-CrossViT for offline handwritten signature recognition and verification," *Complex & Intelligent Systems*, vol. 11, no. 400, pp. 1–15, 2025. 709f1126-b006-40a8-aec3-0d54c37…

[3] A. Hao and L. Zheng, "Detecting AI-generated handwritten signatures using Data-Efficient Image Transformers," *IEEE Transactions on Biometrics, Behavior, and Identity Science*, vol. 7, no. 2, pp. 112–125, 2025.

[4] S. Chattopadhyay, R. Jain, and A. Dey, "Writer-independent handwritten signature verification using Siamese and metric-learning hybrid networks," *Pattern Recognition Letters*, vol. 175, pp. 120–129, 2024.

[5] J. Ren et al., "Two-channel and two-stream transformer framework for offline signature verification," *IEEE Access*, vol. 12, pp. 22456–22468, 2024.

[6] X. Longjam, K. Thongam, and S. Singh, "Hybrid CNN–BiLSTM model for enhanced handwritten signature verification," *Applied Intelligence*, vol. 54, pp. 14800–14815, 2024.

[7] A. Wei, P. Zhang, and F. Guo, "Inverse-discriminative networks for robust offline signature recognition," *Expert Systems with Applications*, vol. 235, pp. 119–144, 2024.

[8] A. Abdirahma, T. Khan, and S. Rehman, "Benchmarking deep learning models for signature verification: MobileNet, Inception, ResNet, and YOLO-based hybrids," *Neural Computing and Applications*, vol. 36, pp. 17845–17860, 2024.

[9] M. Chaudhuri and D. Lopez-Paz, "On the reliability of confidence scores in neural models," *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pp. 11550–11562, 2023.

[10] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," *Advances in Neural Information Processing Systems*, vol. 30, pp. 1–15, 2017.

[11] A. Krishnan and O. Tickoo, "Improving model calibration with temperature scaling across real-world uncertainty," *arXiv preprint arXiv:2006.12315*, 2020